

Memoria Virtuale



Obiettivi

- ❑ Descrivere i benefici di un sistema a memoria virtuale
- ❑ Spiegare i concetti di **demand paging**, **sostituzione di pagina** e l'allocazione di page frames
- ❑ Discutere il principio del **modello working-set**
- ❑ Esaminare come è gestita la **memoria kernel**

Background

- Il codice deve essere in memoria per essere eseguito, ma non tutto è sempre necessario
 - Codice di errore, routine non usate, strutture dati grandi
- Non necessario tutto il codice del programma nello stesso tempo

- Se possibile eseguire programmi parzialmente caricati ...
 - Programmi non più vincolati dalla memoria fisica
 - Ogni programma occupa meno memoria a run-time -> più programmi eseguiti allo stesso tempo
 - ▶ Miglior utilizzo di CPU e maggior throughput senza incremento in tempi di risposta o tempo di turnaround
 - Meno I/O necessari per caricare o swappare programmi in memoria -> ogni programma utente più veloce

Background

- **Memoria Virtuale** – separazione della memoria logica dalla memoria fisica
 - Solo una parte del programma in memoria per l'esecuzione
 - Spazio di indirizzi logici può essere più grande dello spazio di indirizzi fisici
 - Spazio indirizzi condiviso da molti processi
 - Creazione di processi più efficiente
 - Più programmi eseguiti in concorrenza
 - Meno I/O necessario per caricare o swappare processi

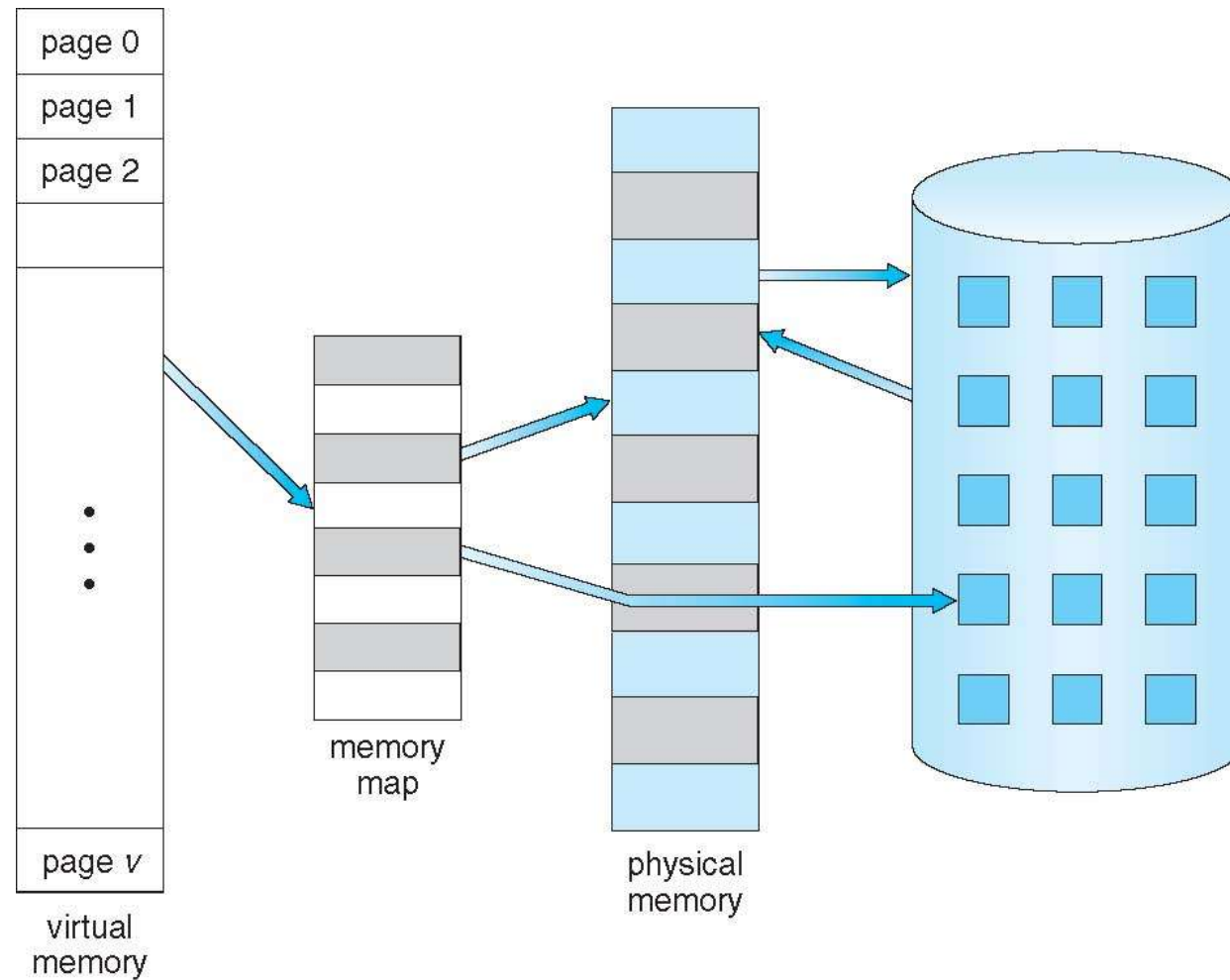
Background

- **Spazio degli indirizzi virtuali** – vista logica su come il processo è contenuto in memoria
 - Spazio inizia da un indirizzo (es. indirizzo 0)
 - Indirizzi contigui fino alla fine dello spazio
 - Memoria fisica organizzata in page frame
 - MMU deve mappare indirizzo logico su fisico

- Memoria Virtuale implementata con:
 - Demand paging
 - Demand segmentation

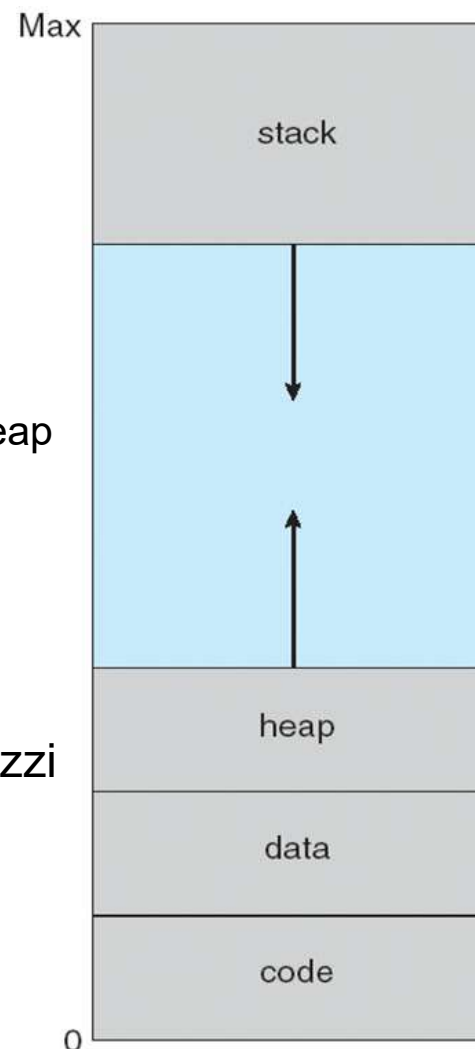
Memoria Virtuale più grande della memoria fisica

Schema che mostra memoria virtuale più grande di quella fisica



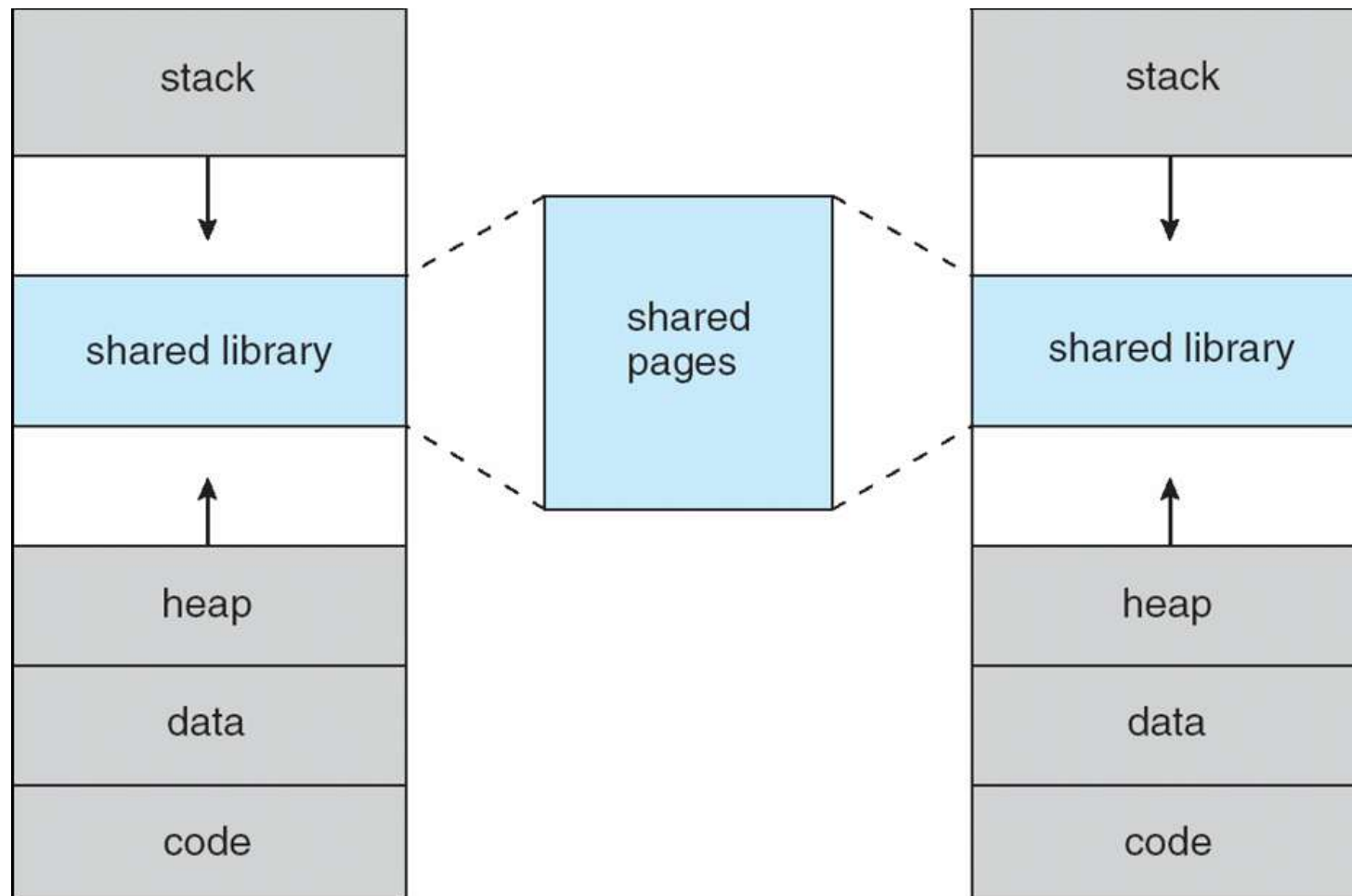
Spazio degli indirizzi virtuali

- Di solito spazio di indirizzi logici con:
 - Stack inizia da indirizzo logico massimo e cresce “in giù”
 - Heap cresce “in su”
 - Massimizza l’uso dello spazio di indirizzi
 - Spazio di indirizzi non usato è un buco
 - ▶ memoria fisica non necessaria finchè lo heap o stack non cresce fino a nuova pagina
- Spazio di indirizzi **sparso** con **buchi** lasciati per crescita, dynamically linked libraries, etc
- Librerie di sistema condivise nello spazio di indirizzi virtuali
- Memoria condivisa mappando pagine read-write nello spazio degli indirizzi virtuali
- Le pagine possono essere condivise durante il `fork()` per accelerare la creazione del processo



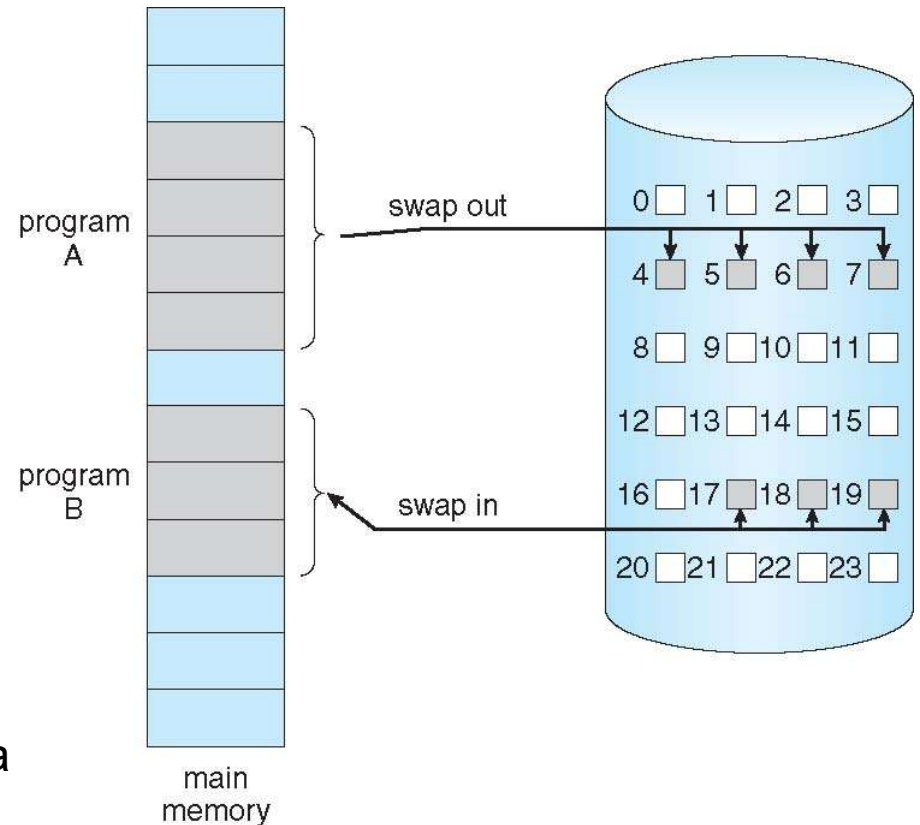
Shared Library usando Memoria Virtuale

- Librerie di sistema condivise nello spazio di indirizzi virtuali
- Shared memory mappando pagine read-write nel virtual address space
- Pagine possono essere condivise durante il fork() per accelerare la creazione del processo



Demand Paging

- Si può portare l'intero processo in memoria a tempo di caricamento
- Oppure portare una pagina in memoria solo quando è necessario:
 - Meno I/O, elimina il non necessario
 - Minor memoria richiesta
 - Risposta più rapida
 - Più utenti contemporaneamente
- Simile al paging con swapping
- Pagina richiesta $\Rightarrow$ riferimento ad essa
 - Riferimento non valido $\Rightarrow$ abort
 - Non-in-memoria $\Rightarrow$ portala in memoria
- **Lazy swapper** – non swappa una pagina in memoria se non richiesta
 - Swapper che carica singole pagine è un **pager**



Concetti di Base

- ❑ Non tutto il processo caricato per l'esecuzione, il pager porta in memoria solo le pagine richieste
- ❑ Come determinare questo insieme di pagine?
 - ❑ Occorre una nuova funzionalità MMU per implementare il demand paging
- ❑ Se la pagina richiesta è già **residenti in memoria**
 - ❑ Non c'è differenza con normale esecuzione
- ❑ Se la pagina richiesta non è residente in memoria
 - ❑ Si genera un trap di page fault
 - ❑ Si deve trovare e caricare la pagina in memoria dallo storage
 - ▶ senza cambiare il comportamento del programma
 - ▶ senza che il programmatore debba cambiare il codice

Valid-Invalid Bit

- Ogni entry della page table ha associato un bit valid–invalid (**v** $\Rightarrow$ in-memory – **memory resident**, **i** $\Rightarrow$ not-in-memory)
- Inizialmente il valid–invalid bit è settato a **i** su tutte le entry
- Es. di uno stato di page table:

Frame #	valid-invalid bit
	v
	v
	v
	i
...	
	i
	i

page table

- Se invalid o il frame non è in memoria o accesso vietato
- Durante la traduzione della MMU se il valid–invalid bit nella page table entry è **i** $\Rightarrow$ page fault

Page Table quando alcune pagine non sono in Memoria

Es. 0 è valido, 1 non è valido, etc.

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

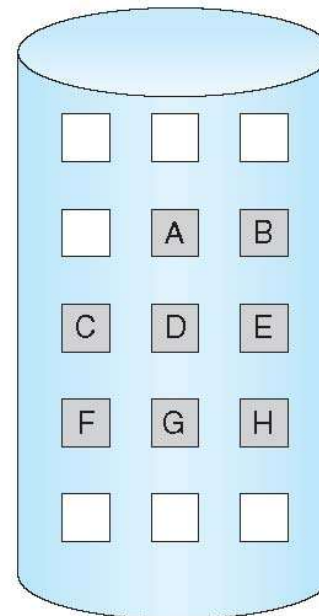
logical
memory

valid-invalid bit		
frame		
0	4	v
1		i
2	6	v
3		i
4		i
5	9	v
6		i
7		i

page table

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

physical memory

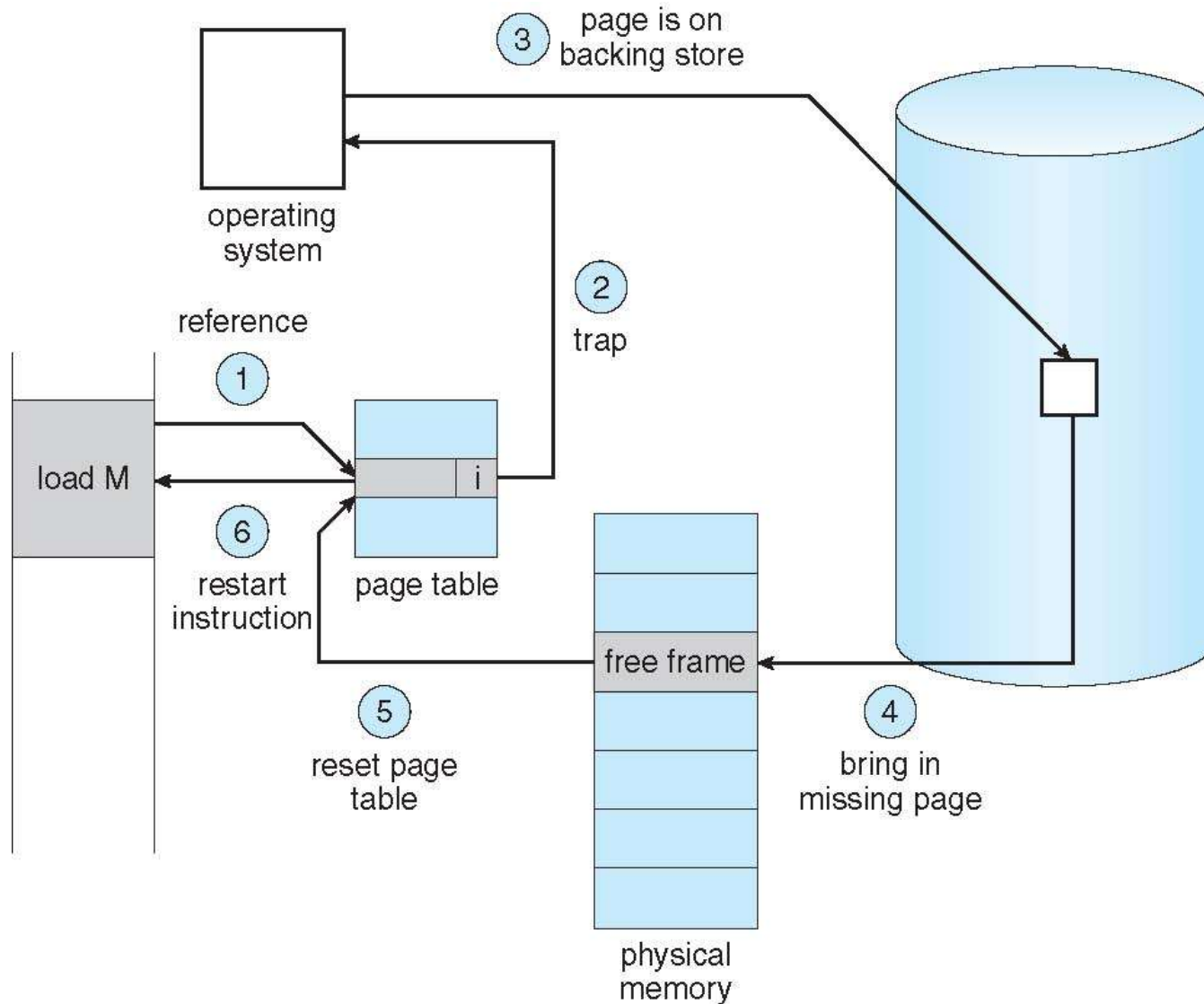


Page Fault

- Il primo riferimento ad una pagina non in memoria genera un trap all'SO:
 - **Trap di page fault**

- Page fault gestito come segue:
 1. SO cerca su altra tabella (in PCB) per verificare se:
 - ▶ Riferimento non valido $\Rightarrow$ abort
 - ▶ Semplicemente non in memoria
 2. Se non valido termina, altrimenti procedi con l'inserimento della pagina
 3. Trova un frame libero (cercando su lista dei frame liberi)
 4. Schedula operazione su disco per lettura della pagina e scrittura nel frame libero
 5. A lettura completata resetta le tabelle per indicare che la pagina è in memoria; setta il validation bit = **v**
 6. Riavvia l'istruzione che ha causato il page fault perché a questo punto il processo può accedere alla pagina in memoria

Passi per Gestire il Page Fault

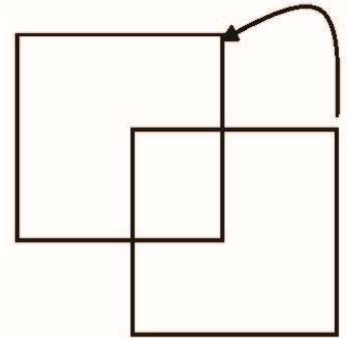


Aspetti del Demand Paging

- Caso estremo – il processo inizia senza pagine in memoria
 - SO setta l'Instruction pointer sulla prima istruzione e la memoria non è residente -> page fault
 - ... lo stesso per ogni altro processo al primo accesso
 - ▶ **Pure demand paging**
- In realtà un'istruzione può accedere più pagine (una per l'istruzione e tante per i dati) -> multipli page fault
 - Es. fetch e decode di istruzione di somma di 2 numeri da memoria che registra il risultato in memoria
 - Problema mitigato dalla **località del riferimento**
- Supporto hardware per demand paging
 - Page table con valid / invalid bit
 - Memoria secondaria (dispositivo di swap con **swap space**)
 - Restart dell'istruzione

Instruction Restart

- ❑ È necessario il restart delle istruzioni dopo il page fault
- ❑ Fault può avvenire in qualunque momento:
 - ❑ fetch, decode, exec
 - ❑ Se interrotta durante il prelievo di operando, va rifatta dal fetch
 - ❑ Si consideri operazione ADD A, B con risultato in C
 - ▶ Prelievo e decodifica di ADD
 - ▶ Prelievo di A
 - ▶ Prelievo di B
 - ▶ Addizione A e B
 - ▶ Memorizzazione somma in C
 - ❑ Se interrotta su C, va ripetuta l'operazione dal prelievo
 - ❑ Istruzioni ancora più complesse:
 - ▶ IBM System 360/370 MVC può spostare 256 byte da locazione ad un'altra anche sovrapposte
 - ▶ se il blocco di dati è tra due pagine e page fault?
 - ▶ Blocco origine modificato, non si può fare restart, due soluzioni:
 - Si controlla prima dell'operazione caricando tutte le pagine
 - Si mantiene in registri lo stato precedente e si ripristina



Instruction Restart

- ❑ È necessario il restart delle istruzioni dopo il page fault
- ❑ Fault può avvenire in qualunque momento:
 - ❑ fetch, decode, exec
 - ❑ Se interrotta durante il prelievo di operando, va rifatta dal fetch
 - ❑ Si consideri operazione ADD A, B con risultato in C
 - ❑ Se interrotta su C, va ripetuta l'operazione dal prelievo
 - ❑ Istruzioni ancora più complesse:
 - ▶ IBM System 360/370 MVC può spostare 256 byte da locazione ad un'altra anche sovrapposte
 - ▶ se il blocco di dati è tra due pagine e page fault?
 - ▶ Blocco origine modificato, non si può fare restart, due soluzioni:
 - Si controlla prima dell'operazione caricando tutte le pagine
 - Si mantiene in registri lo stato precedente e si ripristina
 - ❑ Autoincremento/ autodecremento
 - ▶ Registri autoincrementati/decrementati prima/dopo utilizzo...
 - ▶ ... anche in questo caso, se interrotta l'operazione occorre il ripristino
- ❑ Il paging con demand page richiede un'architettura adatta

Lista dei frame liberi

□ Free Free List

- Quando c'è un page fault occorre caricare la pagina da memoria secondaria in un frame libero
- Il Sistema mantiene una lista di frame liberi



- I frame sono azzerati prima dell'allocazione (zero-fill-on-demand)

Performance del Demand Paging

□ Passi del Demand Paging (caso peggiore)

1. Trap al sistema operativo
2. Salva gli user register e il process state
3. Determina se l'interrupt era un page fault
4. Verifica che il riferimento alla pagina era legale e determina la locazione della pagina su disco
5. Schedula una lettura dal disco ad un frame libero:
 1. Attendi in una coda per il dispositivo finché la richiesta di read è servita
 2. Attendi la ricerca e/o il tempo di latenza
 3. Inizia il trasferimento della pagina al frame libero
6. Mentre attendi alloca la CPU a qualche altro user
7. Ricevi un interrupt dal I/O (I/O completato)
8. Salva i registeri e lo stato del processo dell'altro user
9. Determina che l'interrupt era dal disco
10. Correggi la page table e le altre tabelle per mostrare la pagina in memoria
11. Attendi che la CPU venga di nuovo allocata a questo processo
12. Ripristina gli user register, il process state e la nuova page table, quindi riprendi l'istruzione interrotta

Performance del Demand Paging

- Non sempre tutte le operazioni precedenti sono richieste
- In ogni caso sono richieste tre principali attività:
 - Servi l'interrupt – ben codificato, necessarie centinaia di istruzioni
 - Leggi la pagina – molto tempo
 - Restart del processo – poco tempo
- **Page Fault Rate** $0 \leq p \leq 1$
 - se $p = 0$ non page fault
 - se $p = 1$, ogni riferimento è un fault
- **Effective Access Time (EAT)**

$$\text{EAT} = (1 - p) \times \text{memory access} \\ + p \times (\text{page fault time})$$

Esempio Demand Paging

- Tempo accesso memoria = 200 nanosecondi
- Tempo medio di servizio del page-fault = 8 millisecondi
- $EAT = (1 - p) \times 200 + p (8 \text{ millisecondi})$
 $= (1 - p) \times 200 + p \times 8,000,000$
 $= 200 + p \times 7,999,800$
- Se un accesso su 1000 causa un page fault allora
EAT = 8.2 microsecondi
È un slowdown di un fattore di 40!
- Se si vuole una performance degradation < 10 percent
 - $220 > 200 + 7,999,800 \times p$
 $20 > 7,999,800 \times p$
 - $p < .0000025$
 - < una page fault per ogni 400,000 accessi in memoria

Demand Paging - Ottimizzazioni

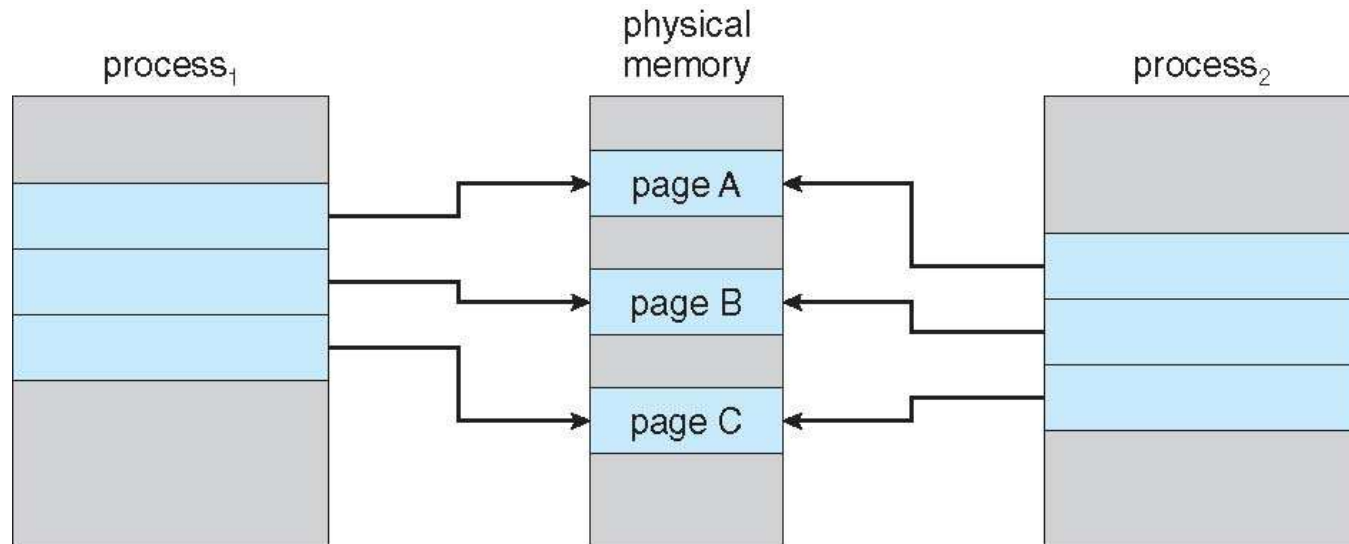
- ❑ I/O su disco per spazio di swap più veloce che quello per accedere al file system anche se su stesso dispositivo
 - ❑ Swap allocato in blocchi molto più grandi minor costo di gestione del file system
 - ❑ Il SO può copiare l'intera immagine del processo su swap space a tempo di caricamento, ma costoso
 - ❑ Altra opzione (Linux, Windows): demand paging da file system all'inizio poi le pagine vanno nello swap space

- ❑ Per limitare lo swap space
 - ❑ File binari eseguibili caricati da file system su frame, ma solo letti, non salvati
 - ▶ Sovrascritti in memoria quando si fa swap out, poi si ricaricano direttamente da disco
 - ❑ Usato in Solaris e nei correnti BSD UNIX
 - ❑ Occorre comunque scrivere nello swap space
 - ▶ Pagine non associate a codice (come stack e heap) – **anonymous memory**
 - ▶ Pagine modificate in memoria principale ma non ancora riscritte sul file system

- ❑ Sistemi Mobile
 - ❑ Tipicamente non supportano swap space
 - ❑ Invece recuperano pagine read-only (come il codice) all'occorrenza (mai anon)

Copy-on-Write

- ❑ La call `fork()` crea duplicato del padre (se il figlio fa subito `exec()` non è necessario il duplicato di memoria)
- ❑ **Copy-on-Write** (COW) permette ai processi padre e figlio di **condividere** inizialmente le stesse pagine in memoria
 - ❑ Le pagine condivise sono marcate come pagine copy-on-write
 - ❑ Se uno dei processi modifica una pagina condivisa allora questa viene copiata
 - ❑ COW permette una creazione di processo più efficiente dal momento che solo le pagine modificate sono copiate
 - ❑ Tecnica usata su Linux, Windows e macOS



Copy-on-Write

- La call `fork()` crea duplicato del padre, se il figlio fa subito `exec()` non necessario duplicato di memoria
- **Copy-on-Write** (COW) permette ai processi padre e figlio di **condividere** inizialmente le stesse pagine in memoria
 - Le pagine sono marcate come pagine copy-on-write
 - Se uno dei processi modifica una pagina condivisa allora questa è copiata
 - COW permette una creazione di processo più efficiente dal momento che solo le pagine modificate sono copiate
 - Tecnica usata su Linux, Windows e macOS
- `vfork()` (virtual memory fork) è variante di `fork()`
 - Sospende il padre e usa l'address space del padre
 - Non usa copy-on-write quindi può scrivere sulla memoria del padre
 - Progettato per avere subito la call `exec()` per il figlio
 - Molto efficiente